

OAK Architecture

AI $\rightarrow$ next step in intelligence

Bottleneck $\rightarrow$ Inadequate learning algorithms

Alberta Plan for AI research

$\downarrow$
next paper

OAK $\Rightarrow$ Agent architecture

OAK $\Rightarrow$ Options And Knowledge

Option is a pair (π, γ)

a way of
behaving

$\rightarrow$ Policy π : States $\rightarrow$ Prob(actions)

a way of $\rightarrow$ Termination condition γ

terminating
the behaviour

$\rightarrow$ States $[0, 1]$

Agent learns a 'high-level transition model' of the world.

Holy grail

- Domain general
 - contains nothing specific to any world
- Experiential
 - grows from runtime experience, not from a special training phase
- Open-ended
 - unlimited in sophistication

Design time & Runtime

Design time $\Rightarrow$ Build domain knowledge into the agent

Run time $\Rightarrow$ Agent learns from experience, & makes plans specific to subset of states encountered

→ Building at design time alone is insufficient

→ If open-ended abstractions are desired, the agent must discover them at runtime

→ everything at runtime : learning, planning, abstraction discovery

Should agent design reflect the world it is expected to be used?

Yes & No

author's quest: No, design should be domain general

Only build meta-methods

Setting: Learn everything at runtime
Entirely experiential

World is more complex than the agent

Everything must be an approximation

→ value functions

→ Policies

→ Transition models

→ States

Thus, world is non-stationary

Runtime learning & planning are
essential.

Thus, find abstractions at runtime

Thus, design time abstractions are
insufficient & deprecated.

Runtime learning always wins over design time

→ RT scales with compute

→ Bitter lesson says above will ~~work~~

→ But, current RT learning doesn't really work

Reward Hypothesis

→ All we need can be thought of as maximization of expected value of the cumulative sum of a received scalar signal (i.e. reward)

Runtime RL arch

model-free: Basic RL $\rightarrow$ Agent constructs
& approx policy and/or
value function, both a
function of the world's
state

non-Markov: Better: Agent constructs
its state representation
(eg: as a feature
vector)

MBRL: Better: constructs transition
model of the
world, & uses
it to plan better
policy

OAK: $\rightarrow$ Agent poses subproblems to obtain
 $\rightarrow$ these enable discovery of higher
features
abstractions

Steps

- Learn policy & value func for maximizing reward (continual deep learning)
- Generate new state features from existing state features
 - Ideas, but no specific sol
- Rechunk features
 - select imp features, pose subproblems
- Create subproblems, one for each highly ranked feature
- Learn solutions to subproblems
- Learn transition models of options & actions
- Plan with transition models.
- Maintain meta data on utility of everything.

Creation of subproblems

Should be part of agent

i = Feature number

R = how intensely we want the feature, balanced against losing reward

Subproblem is to drive the state where the feature is high without losing too much rewards & value

Find an option (π, y) that maximizes value of i th feature at termination while respecting rewards & value

in each state s , maximize

$$E \left[\underbrace{K \phi_i(s_T)}_{\downarrow} + \sum_{R=t+1}^T R_k + \underbrace{\hat{v}(s_T)}_{\downarrow} \mid s_t = s \right]$$

Bonus for stopping when feature is high

approx value of termination state on the main problem

Given a feature, form subproblem

subproblem solving gives a transition model

Transition models (knowledge) plan with
to improve highly adaptive
behaviour.

Mon
b

We need reliable continual deep
learning

Dohare Continual Backprop

IDBP algorithm (Rich Sutton)